

ML REC

1. Implement Naïve Bayes classifier

It is a **Naive Bayes classifier** that assumes features follow a normal (Gaussian) distribution. It calculates probability of each class and picks the highest one. It is a probabilistic classifier based on Bayes theorem that assumes independence between features.

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data = load_iris()
x = data.data[:, :3] # first 3 features
y = data.target      # already numeric
# Split
xtrain, xtest, ytrain, ytest = train_test_split(
    x, y, test_size=0.3, random_state=5
)
# Model
model = GaussianNB()
model.fit(xtrain, ytrain)
# Prediction
y_pred = model.predict(xtest)
print("Accuracy Score:", accuracy_score(ytest, y_pred))
```

2. k-Nearest Neighbor (k-NN)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

data=load_iris()
x=data.data
y=data.target
```

```

x=StandardScaler().fit_transform(x)
xtrain,xtest,ytrain,ytest=train_test_split(x,y,train_size=0.2,random_state=42)
best_k=1
best_acc=0
for k in range(1,11):
    model=KNeighborsClassifier(n_neighbors=k)
    model.fit(xtrain,ytrain)
    acc=accuracy_score(ytest,model.predict(xtest))
    if acc>best_acc:
        best_acc=acc
        best_k=k

final=KNeighborsClassifier(n_neighbors=best_k)
final.fit(xtrain,ytrain)
print("Best k:",best_k)
print("Accuracy Score:",accuracy_score(ytest,final.predict(xtest)))

```

3.Implement Decision trees

ID3:

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
data = load_iris()
X = data.data
y = data.target
# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=293)
# Model (ID3)
model = DecisionTreeClassifier(criterion="entropy",
random_state=0)
model.fit(X_train, y_train)

```

```
# Prediction
y_pred = model.predict(X_test)
print("ID3 Accuracy:", accuracy_score(y_test, y_pred))
# Plot tree
plt.figure(figsize=(15,10))
plot_tree(model, filled=True,
           feature_names=data.feature_names,
           class_names=data.target_names)
plt.title("ID3 Decision Tree")
plt.show()
```

CART

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
# Load dataset
data = load_iris()
x = data.data
y = data.target
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=0)
# CART model (Gini)
model = DecisionTreeClassifier(criterion="gini",
                               random_state=0)
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
print("CART Accuracy:", accuracy_score(y_test, y_pred))
# Plot tree
plt.figure(figsize=(15, 10))
plot_tree(model, filled=True,
           feature_names=data.feature_names,
           class_names=data.target_names)
```

```
plt.title("CART Decision Tree")
plt.show()
```

4.Implement Linear Regression learning algorithm

Linear Regression is a supervised learning algorithm used to model the relationship between input and output variables by fitting a best-fit straight line ($y = mx + c$) using least squares method.

WITHOUT LIBRARIES:

```
import matplotlib.pyplot as plt
x = [1,2,3,4,5]
y = [3,5,7,8,11]
n = len(x)
sx = sum(x)
sy = sum(y)
sxy = sum(x[i] * y[i] for i in range(n))
sx2 = sum(i*i for i in x)

# Slope and intercept
m = (n*sxy - sx*sy) / (n*sx2 - sx*sx)
c = (sy - m*sx) / n

print("Slope:", m)
print("Intercept:", c)
mse = sum((y[i] - yp[i])**2 for i in range(n)) / n
rmse = math.sqrt(mse)
print("MSE:", mse)
print("RMSE:", rmse)
# Predicted values
yp = [m*i + c for i in x]

# Prediction for x = 6
xt = 6
yt = m*xt + c

print("Predicted y for x=6:", yt)
```

```
# Graph
plt.scatter(x, y, label="Data")
plt.plot(x, yp, label="Line")
plt.scatter(xt, yt, marker='*', s=200, label="Prediction")
plt.grid(True)
plt.legend()
plt.show()
```

USING LIBRARIES:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
# Data
X = np.array([1,2,3,4,5]).reshape(-1,1)
y = np.array([2,4,5,4,6])
```

```
# Model
model = LinearRegression()
model.fit(X, y)
```

```
# Output
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```

```
# Prediction
y_pred = model.predict(X)
```

```
# Error
mse = mean_squared_error(y, y_pred)
print("MSE:", mse)
print("RMSE:", np.sqrt(mse))
```

```
# Plot
plt.scatter(X, y)
plt.plot(X, y_pred)
```

```
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Linear Regression")
plt.show()
```

5.Implement Support vector machines learning algorithm- Linear and Non -Linear

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
# Load dataset
data = datasets.load_breast_cancer()
X = data.data[:, :2]
y = data.target
# Train models
linear = SVC(kernel='linear').fit(X, y)
rbf = SVC(kernel='rbf').fit(X, y)
# Function to plot
def plot(model, title):
    plt.scatter(X[:,0], X[:,1], c=y)
    # Create grid
    x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
    y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 100),
        np.linspace(y_min, y_max, 100)
    )
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    # Decision boundary
    plt.contour(xx, yy, Z, levels=[0.5])
    plt.title(title)
# Plot both
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,2,1)
plot(linear, "Linear SVM")
```

```
plt.subplot(1,2,2)
plot(rbf, "Non-Linear SVM (RBF)")
plt.show()
```

6.Implement unsupervised k-means algorithm

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
# Load dataset
data = load_iris()
X = data.data
# Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=42,
n_init=10)
kmeans.fit(X)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
print("Centroids:\n", centroids)
# Plot
plt.scatter(X[:,0], X[:,1], c=labels)
plt.scatter(centroids[:,0], centroids[:,1], marker='X',
s=200)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("K-Means Clustering (Iris)")
plt.show()
```

7.Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same dataset for clustering using k-Means algorithm. Compare

the results of these two algorithms and comment on the quality of clustering.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.datasets import load_iris
from sklearn.metrics import silhouette_score
X = load_iris().data
# K-Means
k_labels = KMeans(n_clusters=3, n_init=10).fit_predict(X)
# EM (GMM)
g_labels = GaussianMixture(n_components=3).fit_predict(X)
# ☒ Calculate scores
k_score = silhouette_score(X, k_labels)
g_score = silhouette_score(X, g_labels)
print("K-Means Score:", k_score)
print("EM Score:", g_score)
# Plot
plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=k_labels)
plt.title("K-Means")
plt.subplot(1,2,2)
plt.scatter(X[:,0], X[:,1], c=g_labels)
plt.title("EM")
plt.show()
# Compare
if g_score > k_score:
    print("EM gives better clustering.")
else:
    print("K-Means gives better clustering.")
```

8.Implement Single Layer Perceptron algorithm

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
```

```

from sklearn.metrics import accuracy_score
data = load_iris()
X = data.data
y = data.target
model = Perceptron()
model.fit(X, y)
y_pred = model.predict(X)
print("Accuracy:", accuracy_score(y, y_pred))
print("Weights:", model.coef_)
print("Bias:", model.intercept_)

```

9. Implement Logistic Regression

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,
mean_squared_error
import numpy as np

X = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)

# 0 = Fail, 1 = Pass
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create model
model = LogisticRegression()
# Train model
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
# Accuracy

```

```

accuracy = accuracy_score(y_test, y_pred)
# Slope and Intercept
print("Slope:", round(model.coef_[0][0], 3))
print("Intercept:", round(model.intercept_[0], 3))
# Accuracy
print("Accuracy:", round(accuracy, 2))
# New prediction
new_data = [[10]]
prediction = model.predict(new_data)
# Display result
if prediction[0] == 1:
    print("Prediction for 10 hours: Pass")
else:
    print("Prediction for 10 hours: Fail")
plt.scatter(X, y)
plt.plot(X, model.predict_proba(X)[: ,1])
plt.show()

```

10. Bagging -

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.ensemble import BaggingClassifier
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
x,y=load_iris(return_X_y=True)
xtr,xt,ytr,yt=train_test_split(x,y,test_size=0.25,random_state=42)
model=BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=10)
model.fit(xtr,ytr)
acc=accuracy_score(yt,model.predict(xt))
print("Accuracy:",round(acc,2))

```

11. Random Forest Algorithmn

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create model
model = RandomForestClassifier(n_estimators=10)

# Train model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", round(accuracy_score(y_test, y_pred),
2))
# New prediction
new_data = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_data)

# Convert number to flower name
print("Prediction:", iris.target_names[prediction[0]])

```

12. Boosting Algorithmn

Adaboost:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create AdaBoost model
model = AdaBoostClassifier(n_estimators=10)

# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", round(accuracy_score(y_test, y_pred),
2))
# New prediction
new_data = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(new_data)
print("Prediction:", iris.target_names[prediction[0]])
```

XGboost:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier

iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create model
model = XGBClassifier(n_estimators=10)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", round(accuracy_score(y_test, y_pred),
2))

# New prediction
new_data = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_data)

print("Prediction:", iris.target_names[prediction[0]])
```